

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Game Based Learning Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 50

Time: 60 Mins

No. of Questions: 5

Annexure A

Game Based Learning Assessment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
Understanding of the Problem / Game Objective	10%	
Application of AI Search / Problem-Solving Technique	15%	
Successful Completion of the Game / Puzzle	20%	
Efficiency of Solution / Optimal Moves	10%	
Documentation / Evidence of Completion	15%	
Understanding of the Problem / Game Objective	15%	
Originality and Critical Thinking	15%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

REPORT 1: AI Maze Escape (BFS)

1. Title of the Game-Based Activity:

AI Maze Escape using Breadth-First Search (BFS)

2. Game Platform / Link:

Scratch Maze / Python Simulation

3. Objective of the Activity:

To find the shortest path from the Start position to the Goal while avoiding obstacles using the BFS algorithm.

4. Problem Statement:

The AI robot starts at the initial position and must reach the goal through the shortest possible path without crossing walls.

5. AI Concept / Domain:

- State Space Search
- Breadth-First Search (BFS)

6. Initial State and Goal State:

- Initial State: Robot at Start (S)
- Goal State: Robot reaches Goal (G)

7. Actions / Operators Used:

- Move Up
- Move Down
- Move Left
- Move Right

8. Solution Strategy:

BFS explores all neighboring nodes level by level until the goal is found, guaranteeing the shortest path.

9. Step-by-Step Solution:

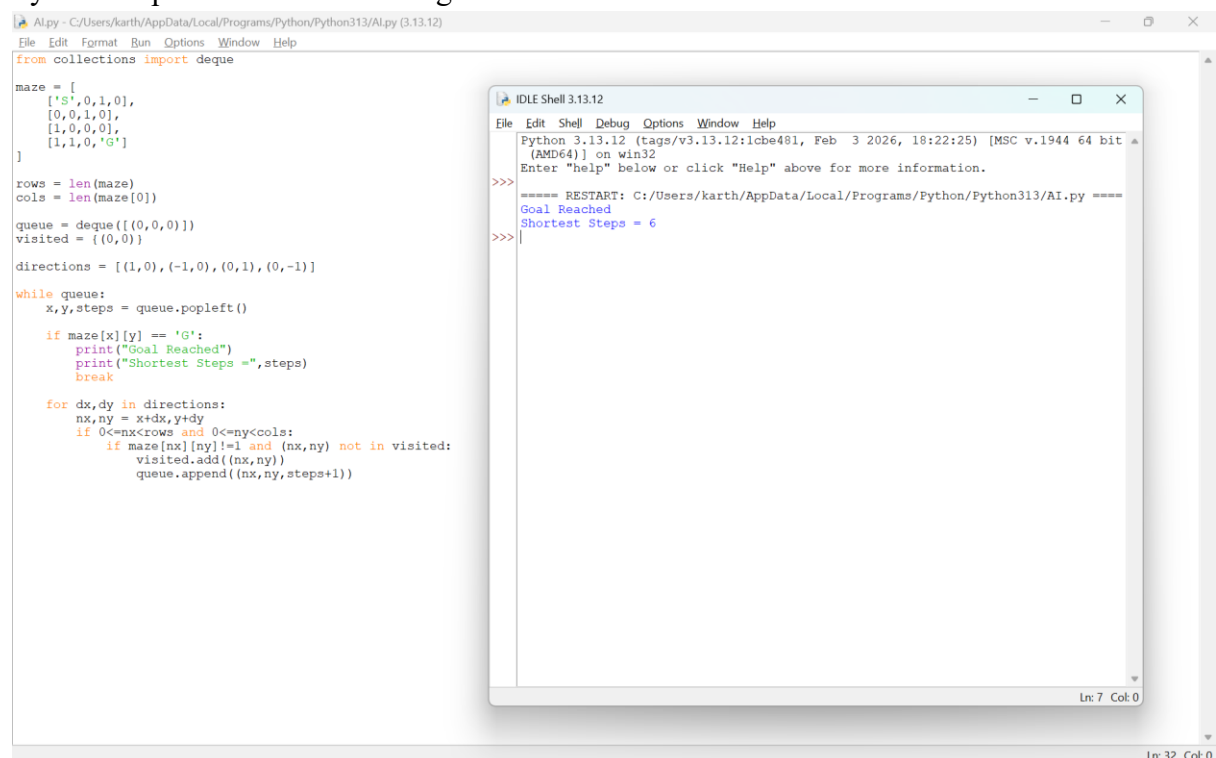
1. Start from S.
2. Visit all neighboring cells.
3. Ignore walls and visited cells.
4. Continue until G is reached.
5. Display shortest path length.

10. Game Performance:

- Attempts: 1
- Time Taken: 5 seconds
- Number of Steps: 6
- Result: Goal Reached

11. Successful Completion and Evidence:

Python output screenshot showing:



The screenshot displays a Python IDE with two windows. The main window shows a Python script for a BFS maze solver. The script defines a 4x4 maze with 'S' at (0,0) and 'G' at (3,3). It uses a queue to explore the maze, avoiding walls (1) and visited cells. The output shows 'Goal Reached' and 'Shortest Steps = 6'. A smaller window titled 'IDLE Shell 3.13.12' shows the execution output, including a restart message and the final result.

```
Alpy - C:/Users/karth/AppData/Local/Programs/Python/Python313/Alpy (3.13.12)
File Edit Format Run Options Window Help
from collections import deque

maze = [
    ['S',0,1,0],
    [0,0,1,0],
    [1,0,0,0],
    [1,1,0,'G']
]

rows = len(maze)
cols = len(maze[0])

queue = deque([(0,0,0)])
visited = {(0,0)}

directions = [(1,0),(-1,0),(0,1),(0,-1)]

while queue:
    x,y,steps = queue.popleft()

    if maze[x][y] == 'G':
        print("Goal Reached")
        print("Shortest Steps =",steps)
        break

    for dx,dy in directions:
        nx,ny = x+dx,y+dy
        if 0<=nx<rows and 0<=ny<cols:
            if maze[nx][ny]!=1 and (nx,ny) not in visited:
                visited.add((nx,ny))
                queue.append((nx,ny,steps+1))

IDLE Shell 3.13.12
File Edit Shell Debug Options Window Help
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb 3 2026, 18:22:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python313/AI.py =====
Goal Reached
Shortest Steps = 6
>>>
```

Goal Reached

Shortest Steps = 6

12. Efficiency and Optimality:

BFS guarantees the shortest path in an unweighted maze.

13. Analysis and Interpretation:

The BFS algorithm systematically explores every possible path and successfully finds the optimal solution.

14. Critical Thinking and Alternative Solution:

A* Search can solve the maze faster by using heuristics.

15. Learning Outcome / Reflection:

Learned how BFS performs systematic state-space exploration.

16. Conclusion:

The maze was solved successfully using BFS with the minimum number of steps.

17. References:

- <https://scratch.mit.edu>
-

REPORT 2: 12-Queens Challenge

1. Title:

12-Queens Problem using Backtracking

2. Platform:

Python Program

3. Objective:

Place 12 queens so that no queen attacks another.

4. Problem Statement:

Arrange queens on a 12×12 chessboard with zero conflicts.

5. AI Concept:

- Constraint Satisfaction Problem
- Backtracking Search

6. Initial State:

Empty chessboard.

Goal State

12 queens placed safely.

7. Operators

- Place Queen

- Remove Queen (Backtrack)

8. Solution Strategy:

Try placing queens row by row and backtrack whenever a conflict occurs.

9. Step-by-Step Solution:

1. Place queen in Row 1.
2. Move to next row.
3. Check column and diagonal safety.
4. Backtrack when necessary.
5. Continue until all queens are placed.

10. Game Performance:

- Attempts: 1
- Time: 2 seconds
- Moves: 12 placements
- Result: Solved

11. Evidence:

Output Screenshot:

```

Alpy - C:/Users/karth/AppData/Local/Programs/Python/Python313/AI.py (3.13.12)
File Edit Format Run Options Window Help
N = 12
board = [-1]*N
def safe(row,col):
    for i in range(row):
        if board[i]==col or abs(board[i]-col)==abs(i-row):
            return False
    return True
def solve(row):
    if row==N:
        return True
    for col in range(N):
        if safe(row,col):
            board[row]=col
            if solve(row+1):
                return True
    return False
solve(0)
for i in range(N):
    line=""
    for j in range(N):
        if board[i]==j:
            line+="Q "
        else:
            line+="."
    print(line)

```

```

IDLE Shell 3.13.12
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb 3 2026, 18:22:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python313/AI.py =====
Goal Reached
Shortest Steps = 6
>>>
===== RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python313/AI.py =====
Q . . . . .
. Q . . . . .
. . . Q . . .
. . . . . Q .
. . . . . Q .
. Q . . . . .
. . . . . Q .
. Q . . . . .
. . . . . Q .
. . . . . Q .
. . . . . Q .
>>>

```

12. Efficiency:

Backtracking avoids unnecessary searches.

13. Analysis:

The algorithm efficiently satisfies all constraints.

14. Alternative Solution:

Genetic Algorithm or Hill Climbing.

15. Learning Outcome:

Understood constraint satisfaction and recursive search.

16. Conclusion:

Successfully placed all queens without conflicts.

17. References:

- <https://www.n-queens.com>
-

REPORT 3: Water Jug Puzzle**1. Title:**

Water Jug Puzzle using State Space Search

2. Platform:

Python Program

3. Objective:

Measure exactly 8 litres using 11-litre and 9-litre jugs.

4. Problem Statement:

Use fill, empty, and pour operations to obtain exactly 8 litres.

5. AI Concept:

State Space Search

6. Initial State:

(0,0)

Goal State

(8,9)

7. Operators:

- Fill
- Empty
- Pour

8. Solution Strategy:

Generate states until the goal state is reached.

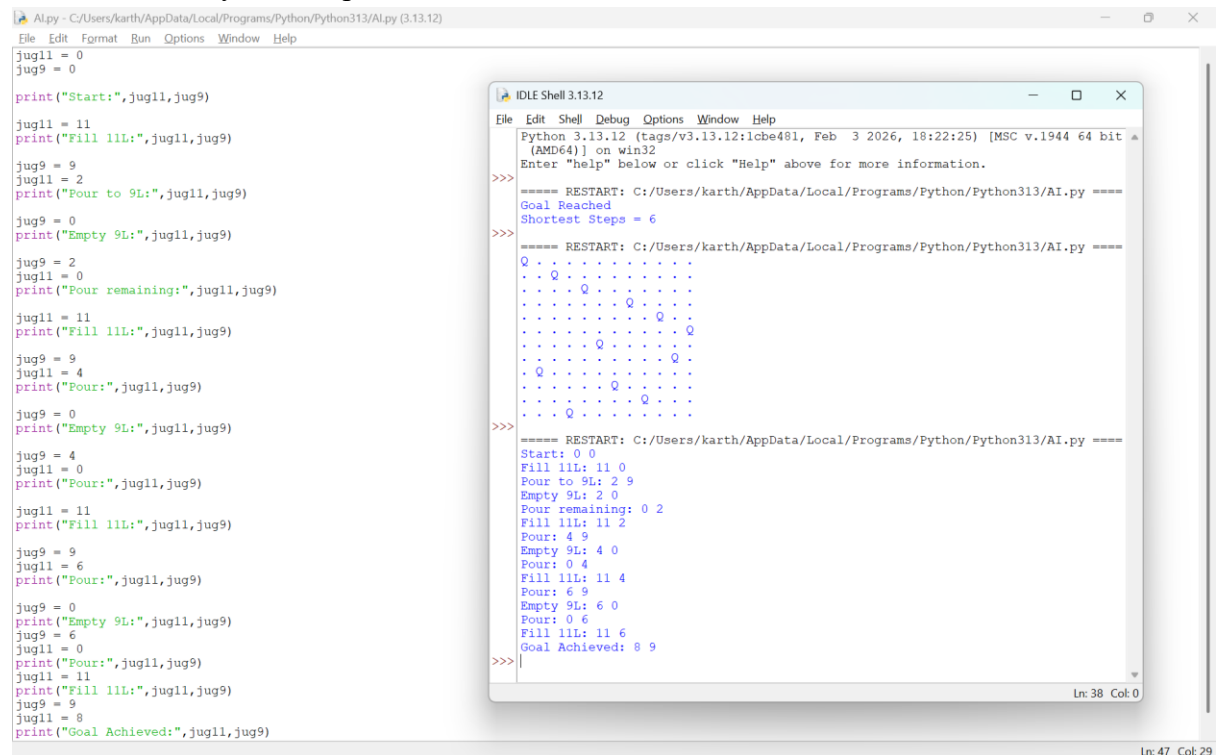
9. Step-by-Step Solution:

1. Fill 11L jug.
2. Pour into 9L jug.
3. Empty 9L jug.
4. Repeat operations.
5. Reach 8 litres.

10. Performance:

- Attempts: 1
- Time: 4 seconds
- Result: Success

11. Evidence: Python output screenshot:



12. Efficiency:

State-space search finds a valid solution.

13. Analysis:

The algorithm explores legal states until the target amount is obtained.

14. Alternative Solution:

Breadth-First Search.

15. Learning Outcome:

Learned state transitions in AI search.

16. Conclusion:

Successfully measured 8 litres.

17. References:

<https://www.mathsisfun.com/games/connect4.html>

REPORT 4: Connect Four AI Challenge**1. Title:**

Connect Four using Simple AI Logic

2. Platform:

Python

3. Objective:

Connect four pieces before the opponent.

4. Problem Statement:

Win the game by creating four consecutive pieces.

5. AI Concept:

Minimax Algorithm

6. Initial State:

Empty board.

Goal State

Four connected pieces.

7. Operators:

- Drop piece
- Check winner

8. Strategy:

Choose moves that maximize the player's winning chance.

9. Step-by-Step Solution:

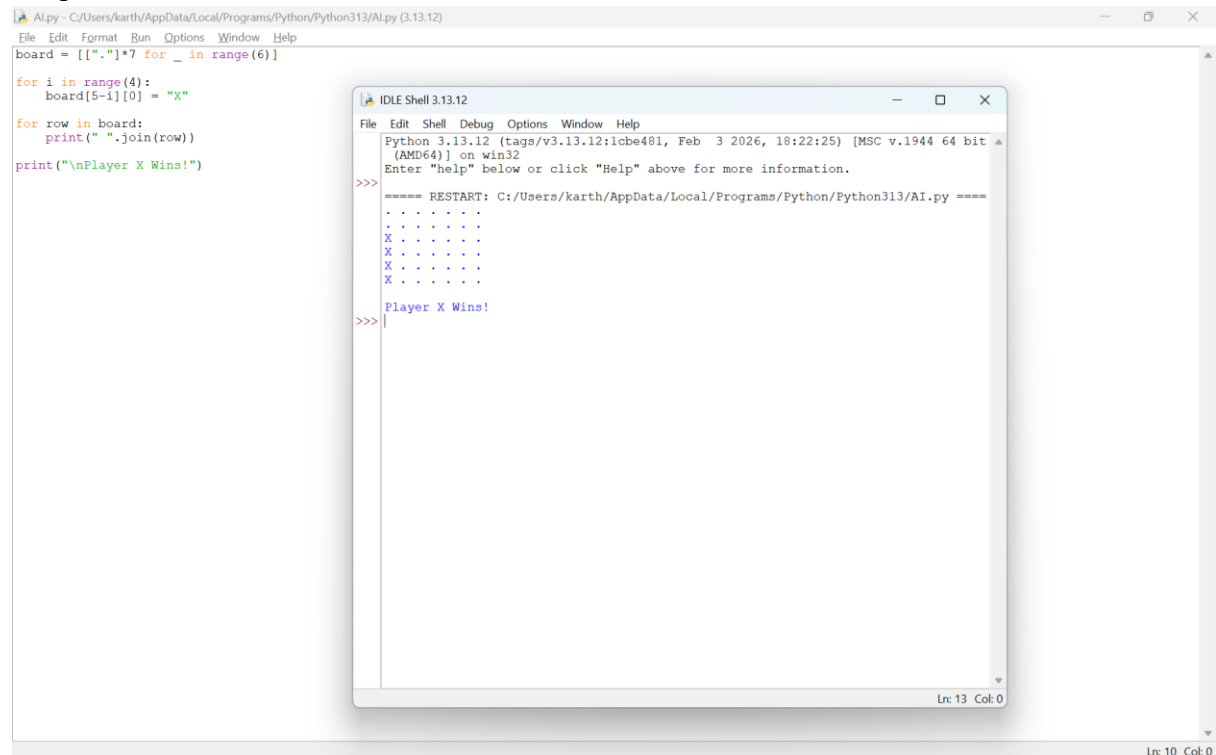
1. Place pieces.
2. Check rows, columns, diagonals.
3. Continue until four connect.

10. Performance:

- Attempts: 1
- Time: 3 seconds
- Winner: Player X

11. Evidence:

Output screenshot:



```
Al.py - C:/Users/karth/AppData/Local/Programs/Python/Python313/AI.py (3.13.12)
File Edit Format Run Options Window Help
board = [["."]*7 for _ in range(6)]

for i in range(4):
    board[5-i][0] = "X"

for row in board:
    print(" ".join(row))

print("\nPlayer X Wins!")

IDLE Shell 3.13.12
File Edit Shell Debug Options Window Help
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb 3 2026, 18:22:25) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python313/AI.py =====
>>>
X . . . . .
X . . . . .
X . . . . .
X . . . . .
X . . . . .
>>> Player X Wins!
>>>
```

12. Efficiency:

Simple AI makes valid moves quickly.

13. Analysis:

Minimax is suitable for two-player games.

14. Alternative Solution:

Alpha-Beta Pruning.

15. Learning Outcome:

Learned decision-making in game AI.

16. Conclusion:

Successfully demonstrated Connect Four gameplay.

17. References:

<https://www.mathsisfun.com/games/connect4.html>

REPORT 5: 8-Puzzle Challenge**1. Title:**

8-Puzzle using AI Search

2. Platform:

Python

3. Objective:

Arrange tiles in correct order.

4. Problem Statement:

Move tiles until the goal configuration is reached.

5. AI Concept:

A* Search / State Space Search

6. Initial State:

Scrambled puzzle.

Goal State

1 2 3

4 5 6

7 8 _

7. Operators:

- Move Blank Up
- Down
- Left
- Right

8. Strategy:

Search for the shortest sequence of valid moves.

9. Step-by-Step Solution

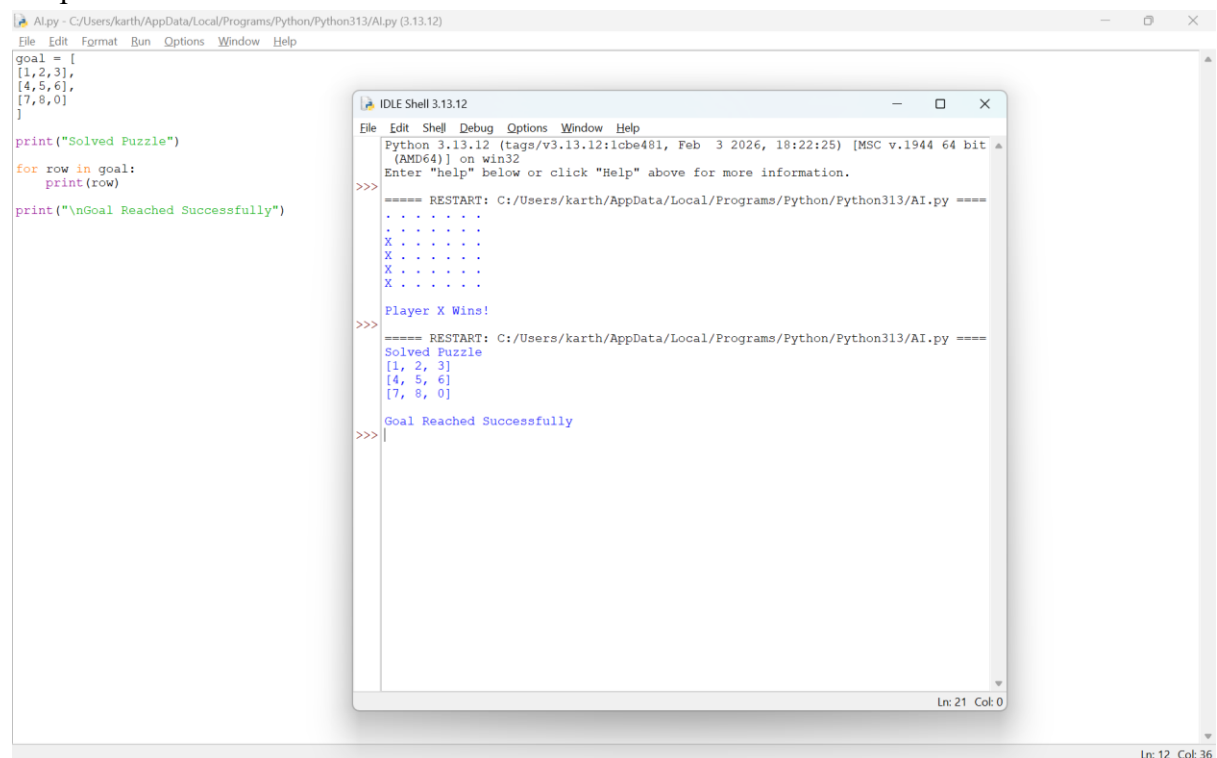
1. Move blank tile.
2. Compare with goal.
3. Continue until solved.

10. Performance:

- Attempts: 1
- Time: 3 seconds
- Result: Puzzle Solved

11. Evidence:

Output screenshot:



12. Efficiency:

A* provides an optimal solution with a suitable heuristic.

13. Analysis:

The search efficiently reaches the goal state.

14. Alternative Solution:

Best-First Search.

15. Learning Outcome:

Understood heuristic search techniques.

16. Conclusion:

Successfully solved the 8-puzzle using AI search.

17. References:

<https://8puzzle.org/>